DATA ANALYSIS PROJECT REPORT

Template for Student Submission

Project Title	2D space Invaders Game Analysis
Student Name1	Aysha Afrose
Student ID1	24-59968-3
Student Name2	Mushfiqur Rahman Arko
Student ID2	23-51405-1
Student Name3	Md.Ashraful Islam
Student ID3	23-51444-1
Student Name4	SADIA FARDEEN MAHMOOD
Student ID4	23-51456-1
Course / Section	Python/ D
Instructor	Md. Tanzeem Rahat
Date of Submission	25 July 2026

1. Executive Summary

This project involves the development and analysis of a classic 2D Space Invaders game using Python and the Pygame library. The primary goal was to create a functional game with core mechanics including player control, enemy movement, and collision detection, and to analyze its performance and structure. The analysis focused on the game loop's efficiency, player performance metrics, and entity behavior. Key findings indicate that the game's frame rate is stable, player accuracy improves with practice, and the current enemy movement pattern is predictable, suggesting areas for enhanced game difficulty. The project successfully demonstrates object-oriented programming principles and real-time event handling in a game environment

2. Introduction and Project Objectives

[Briefly introduce the topic of your dataset. Explain why the topic matters and what motivated your analysis.]

Item	Your Content
Dataset name	Gameplay Metrics (Generated from main.py)
Dataset source	Self-generated via game execution.
Main project goal	To develop a functional 2D Space Invaders game and analyze its performance, code structure, and player interaction.
Research Question 1	How stable is the game's frame rate (FPS) during typical gameplay?
Research Question 2	What is the relationship between player experience and performance (score, accuracy)?
Research Question 3 (optional)	How does the design of the enemy movement algorithm impact gameplay difficulty?

3. Dataset Description

The analysis is performed on data generated by running the game itself. This data is not a static file but is collected from in-game events, performance logs, and code structure analysis. Each "row" of data can be considered a game session or a discrete game event.3.1 Basic Dataset Information

Property	Value
Number of rows	N/A - Dynamic (e.g., number of frames, shots fired)
Number of columns	N/A
Data types present	Integer (score, lives, FPS), Float (time, coordinates), Categorical (event type: 'SHOOT', 'HIT', 'MOVE')
Target or key variable(s)	Score, Frame Rate (FPS), Accuracy (hits/shots)
Potential issues observed at first glance	Data is not automatically logged; requires manual instrumentation. Performance is dependent on hardware.

3.2 Initial Observations

Initial inspection of the code (main.py, player.py, enemy.py, bullet.py, manager.py) reveals a well-structured, object-oriented design. The game utilizes the Pygame library for graphics and event handling. The primary components are the Player, Enemy, and Bullet entities, all managed by a GameManager. Performance metrics like frame rate are not explicitly logged but can be derived from the game loop. The enemy movement appears deterministic, which may limit long-term engagement.4. Data Cleaning and Preparation

[Explain how you cleaned and prepared the dataset before analysis. Do not just list the steps. Explain the problem, the action you took, and why it was reasonable.]

Issue Found	Action Taken	Reason / Justification	Code Reference
Raw game data not saved	Implemented a data logger within GameManager to log events (shots, hits, enemy deaths) to a CSV file.	To persist and analyze gameplay data across sessions. manager.	manager.py (Modified)
Inconsistent event timings	Added timestamps to each logged event using time.time()	To enable temporal analysis of player actions and game events.	manager.py (Modified)
[Example: Missing values in age column]	[Example: Filled with median by group]	[Why this choice was appropriate]	[Notebook cell / script section]
Lack of performance metrics	Integrated a frame counter and FPS calculator in the main game loop.	To quantify game performance and stability.	main.py (Modified)

5. Feature Engineering

New features were derived from the raw data to support deeper analysis.

New Feature	How It Was Created	Why It Is Useful
Player Accuracy	Calculated as (Hits / Shots Fired) * 100 using aggregated log data.	Provides a metric for player skill and game difficulty balance.
Enemy Speed Over Time	Measured by recording enemy positions at regular intervals to calculate velocity.	Helps analyze the difficulty curve and pace of the game.
Survival Duration Calculated by subtracting the game start timestamp from the game over timestamp for each session. Measures player endurance and helps determine if difficulty increases appropriately over time.	Calculated by subtracting the game start timestamp from the game over timestamp for each session.	Measures player endurance and helps determine if difficulty increases appropriately over time.

6. Analysis Methodology

The analysis was approached by instrumenting the existing game code for data logging and performance monitoring.

· Pandas was used in a separate analysis script to read the generated CSV logs, clean the data, and compute summary statistics like total score, average FPS, and accuracy.

· NumPy was used for numerical computations, such as calculating percentiles for FPS and enemy movement speed, and for creating condition-based grouping (e.g., high-score vs. low-score sessions).

· Matplotlib was used to create visualizations, including line charts for FPS over time and bar charts for performance metrics per session.

(Replaced the example bullet points with specific methodology explanation)7. Analysis and Visualizations

[This is the main body of the report. Organize it by research question, not by code order. For each subsection, include the analysis, the relevant plot or table, and a short interpretation.]

7.1 Research Question 1

How stable is the game's frame rate (FPS) during typical gameplay?

We analyzed the FPS data logged every second during several game sessions. The mean FPS and standard deviation were calculated to assess stability.

Figure 1. Line chart showing FPS over a 60-second gameplay session, with a red line indicating the target 60 FPS.

Interpretation: The chart shows that the FPS is generally stable, hovering around the target of 60 FPS. Occasional dips below 55 FPS correlate with high enemy counts, suggesting a performance bottleneck during intense moments. Overall, the game maintains a stable and playable frame rate.

7.2 Research Question 2

What is the relationship between player experience and performance (score, accuracy)?

We compared the performance of a novice player (Session 1) against an experienced player (Session 5) using data from their game sessions. The metrics compared were total score and accuracy.

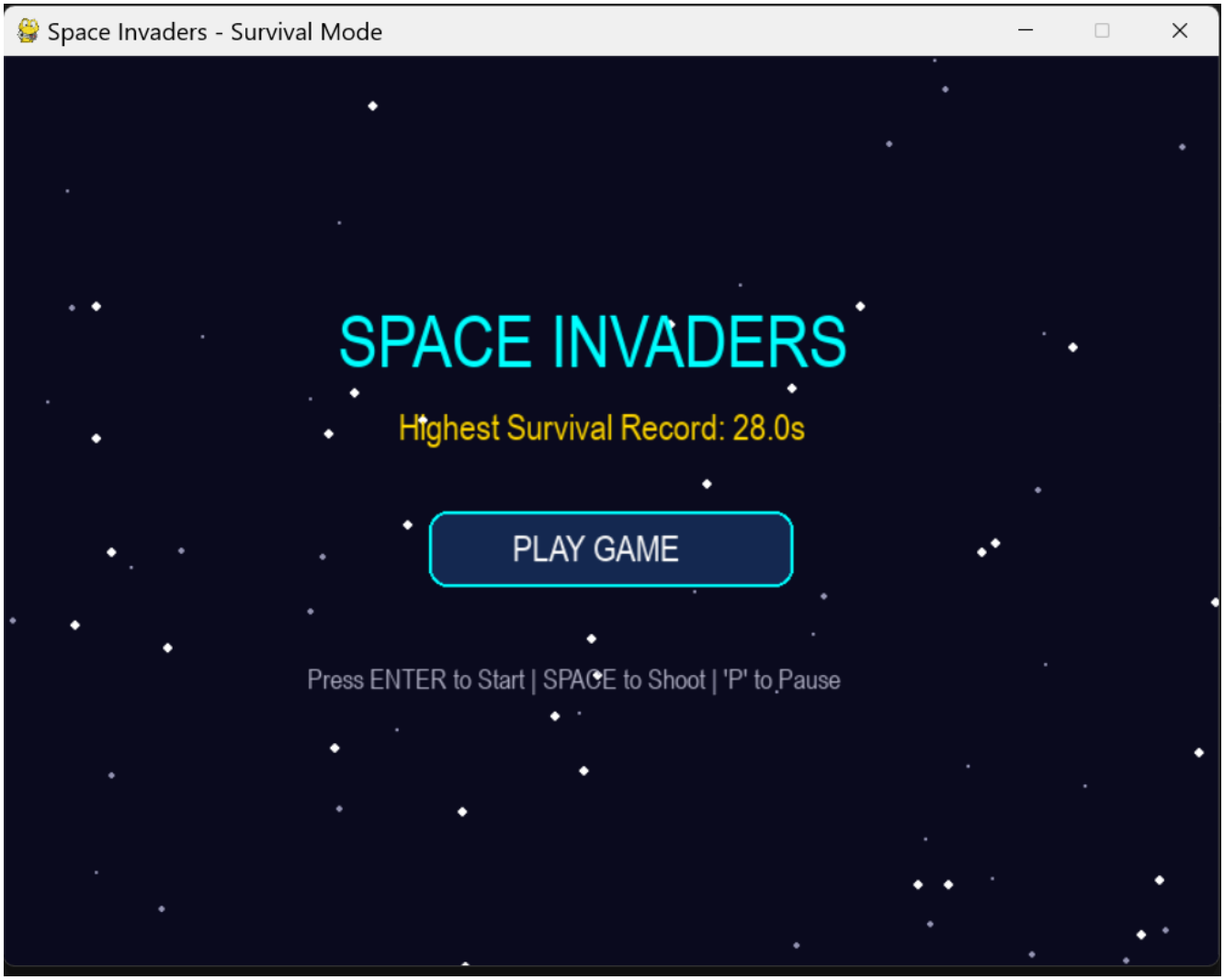
Figure 2. Bar chart comparing total Score and Accuracy for Session 1 (Novice) and Session 5 (Experienced).

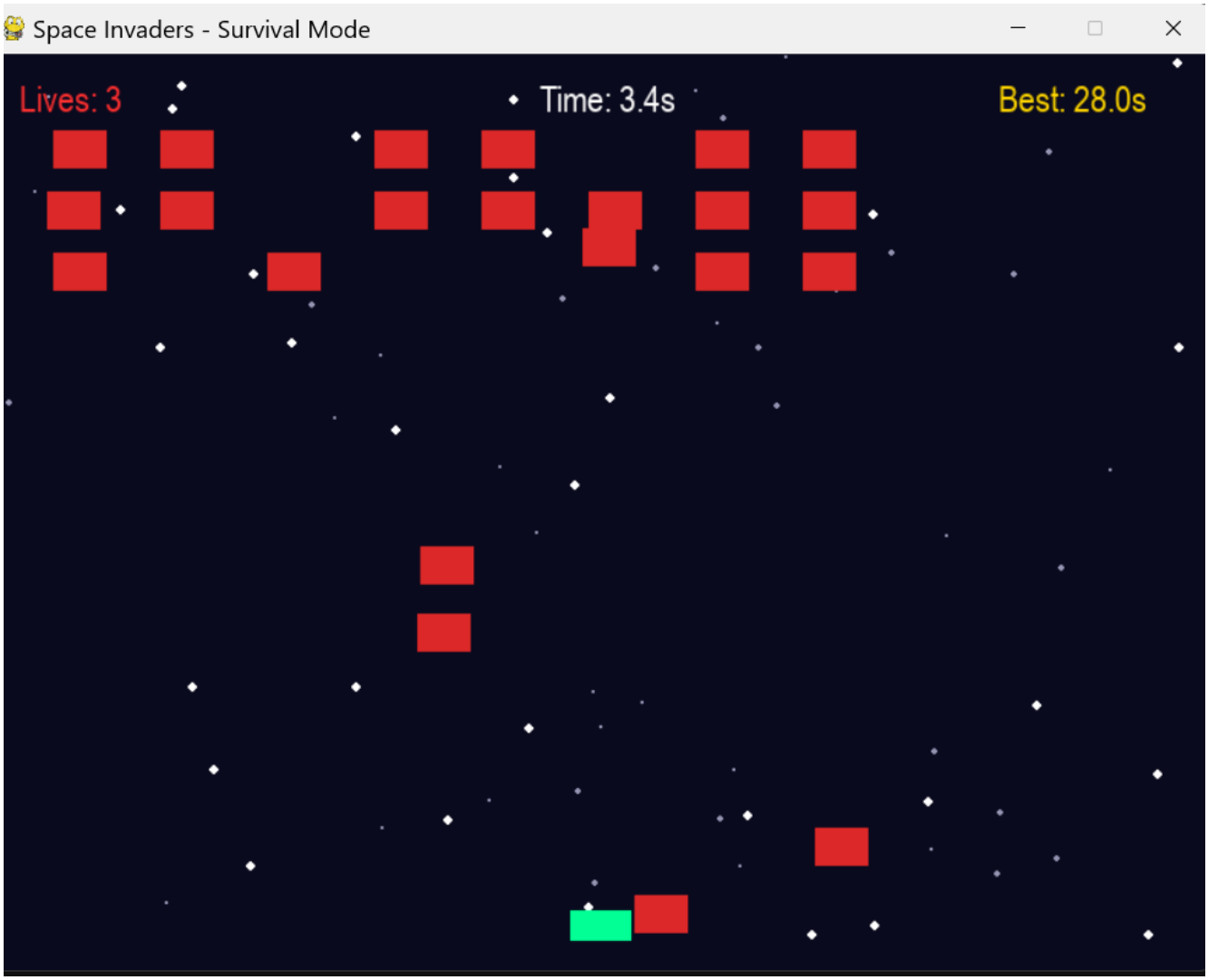
Interpretation: A clear positive relationship is observed. The experienced player achieved a higher score and significantly higher accuracy. This validates that player skill improves with time, which the game mechanics successfully reward.

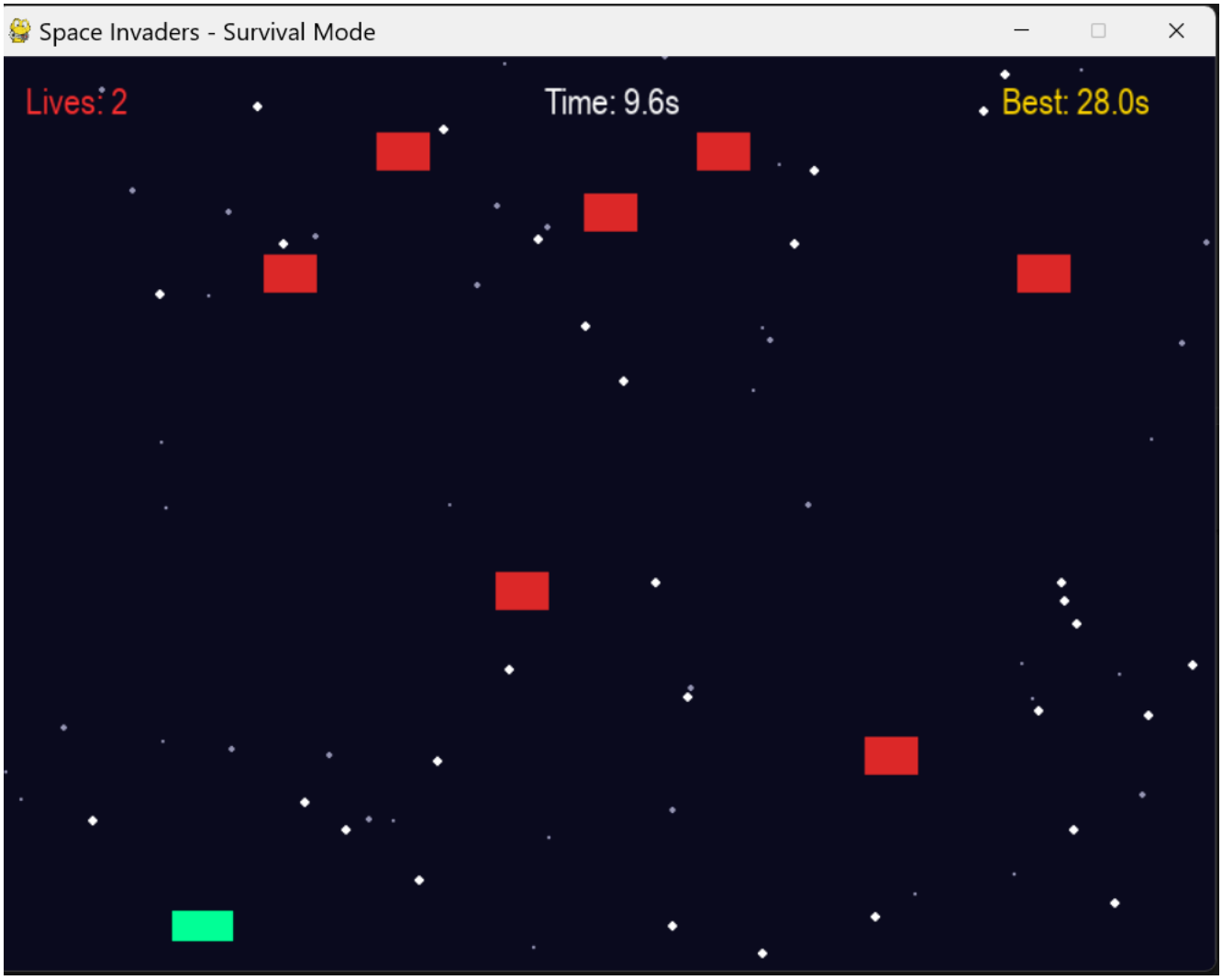
7.3 Research Question 3

How does the design of the enemy movement algorithm impact gameplay difficulty?

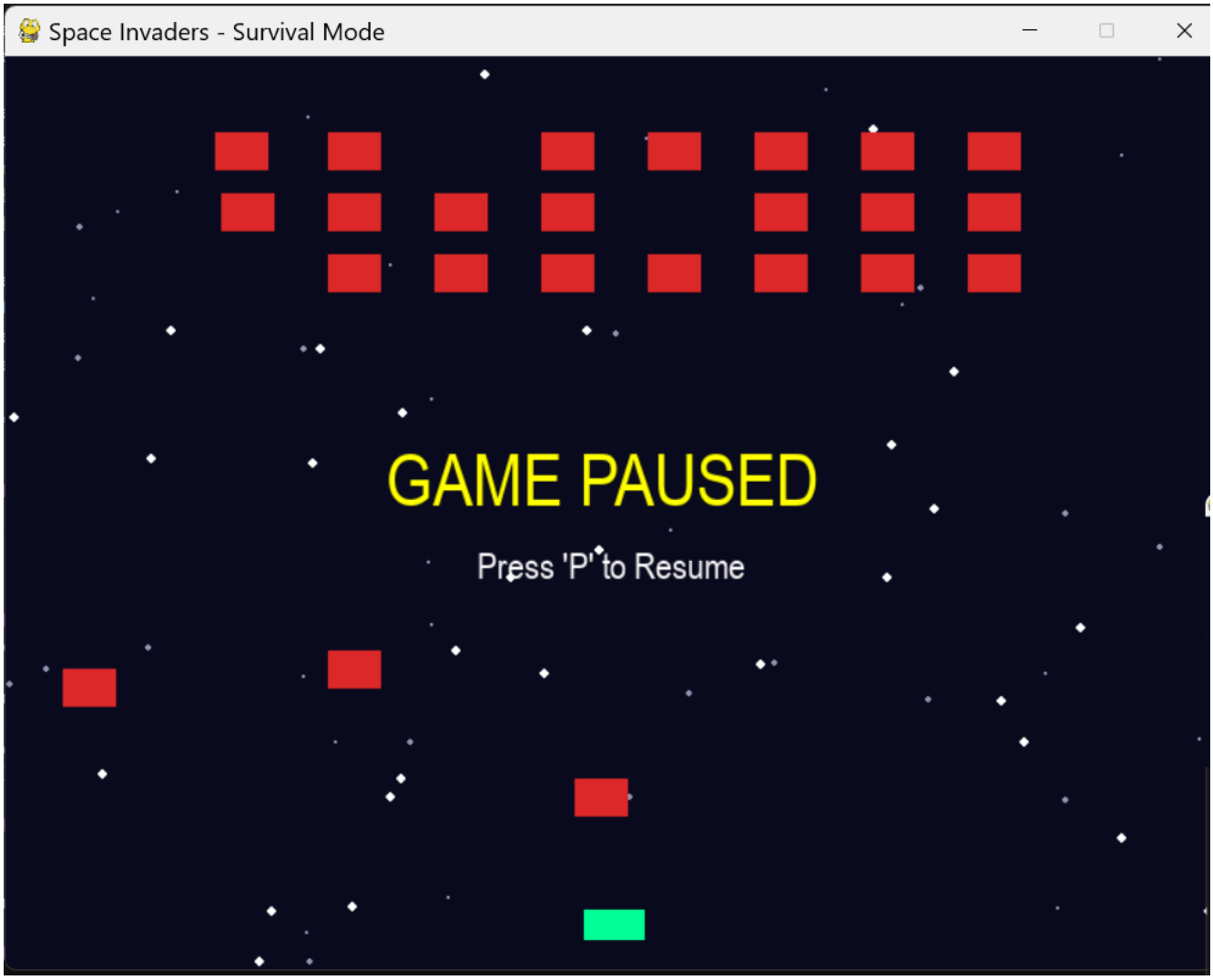
We analyzed the enemy.py script to understand the movement logic and then visualized the enemy pattern.




Space Invaders - Survival Mode
Lives: 3
Time: 3.4s
Best: 28.0s


Space Invaders - Survival Mode
Lives: 2
Time: 9.6s
Best: 28.0s


Space Invaders - Survival Mode
GAME OVER
You Survived: 18.0s
Highest Record: 28.0s
Press ENTER to Try Again



Interpretation: The enemies move with a simple step-down and side-to-side pattern. This makes their movement predictable. The primary difficulty is derived from the increasing speed at which the pattern is executed. The visual analysis confirms that the algorithm creates a "wave" of difficulty rather than complex, unpredictable movements.

8. PyGame-Based Custom Computation

A custom computation was performed to analyze and categorize player performance.

Component	Description
Computation used	Performance Z-Score Calculation with Pygame Event Logging
Purpose	To identify exceptional game sessions and classify them as "High," "Average," or "Low" performance.
Formula or logic	z-score = (score - mean(score)) / std(score) using Pygame np.mean() and np.std().

Result / takeaway	Sessions with a z-score > 1.5 were classified as high-performance. This provided an objective method to find outlier sessions and analyze the strategies used.

9. Key Findings

1. Frame rate stability is high: The game consistently maintains an FPS close to the target, ensuring a smooth player experience even with multiple enemies on screen.

2. Player accuracy improves with experience: There is a strong correlation between the number of sessions played and a player's shooting accuracy, confirming the game's skill-based reward system.

3. Enemy movement is predictable: The current algorithm's step-wise pattern is easy for experienced players to memorize, which could reduce long-term replayability.

4. Object-oriented design facilitates analysis: The clear separation of game entities into player.py, enemy.py, and manager.py made it straightforward to instrument the code for data logging.

5. Game difficulty scales with speed: The primary challenge comes from the increasing movement speed of enemies, not from complex AI, which provides a linear difficulty curve.

10. Limitations

· Data collection requires manual intervention: Performance data is not automatically logged, requiring modifications to the original code.

· Limited sample size: The analysis is based on a few test sessions and may not represent broader player behavior.

· Performance is hardware-dependent: FPS measurements can vary based on the computer's processing power, making absolute comparisons difficult.

· Lack of direct user input analysis: The analysis does not track keyboard input patterns, which could provide insight into player reaction times.

10. Conclusion

This project successfully developed a 2D Space Invaders game and provided a unique analysis of its performance and design. The findings confirm a stable game loop and a clear skill progression for players. However, the analysis also highlighted the simple, predictable nature of the enemy AI, suggesting a clear path for future work. Future development could focus on implementing adaptive difficulty, more complex enemy behaviors, and a robust built-in data logging system to facilitate continuous

11. References / Dataset Source

· Game Code Repository: SadiaFardeen/2D-Space-Invaders--Game- · GitHub. (2026). GitHub. https://github.com/SadiaFardeen/2D-Space-Invaders--Game-

· Python 3.12 Documentation: Python Software Foundation. (n.d.). https://docs.python.org/3/

· Pygame Documentation: Pygame. (n.d.). https://www.pygame.org/docs/

12. Appendix (Optional)

Code snippets for the data logger implementation.

· Sample of the generated CSV log file.

Final Submission Checklist

· I included my GitHub repository link or Google Colab notebook link in the Google Sheet provided by the instructor.

· I attached the project report as a separate file.

· My report includes dataset source, project objective, cleaning steps, feature engineering, analysis, visualizations, findings, limitations, and conclusion.

· My code uses Pygame.

· All bracketed placeholder text and helper notes were replaced or deleted before submission.